

Human Value Exchange

Hermes Agent Update Brief

Potential new features and architectural opportunities for HVE Life OS

Prepared for	Luna, HVE technical architecture
Prepared by	Hans Westphal with M365 Copilot synthesis
Date	17 Aug 2026
Purpose	Translate Nous Research Hermes updates into concrete HVE feature candidates, architecture prompts, and next decisions.

Executive signal

Hermes is moving from a single prompted agent into a persistent, multi-agent operating layer. For HVE, the practical opportunity is to turn Hermes from a coding helper into a local-first workforce that can monitor, build, audit, report, and coordinate across the HVE five types of wealth while preserving sovereignty and portability.

Recommendation: treat this as a design review input for Luna, not an immediate adoption mandate. Prioritize small, reversible experiments on Mercury/Hermes before coupling core HVE workflows to new Hermes behaviours.

1. Why this matters for HVE

The Aug 10 to 16 Hermes update pattern lines up strongly with HVE Life OS: persistent agents, local-first automation, tool use, delegated workers, richer plugins, browser/API learning, and multi-machine orchestration. This is exactly the operating model HVE has already been moving toward: M365 Copilot thinks, Luna designs, Hermes builds, Mercury proves, and Hans decides.

Design principle for Luna

Every candidate below should be evaluated against the HVE Prime Directive: does it increase sovereignty, run locally where possible, remain portable, and reduce dependency rather than create a fragile cloud-first stack?

2. High-level readout of the Hermes shift

- From profile to bot workforce: profiles can become named, specialised agents with memory, tools, skills, routines, and identity.
- From one-shot tasks to responsibility loops: /loop points toward recurring audits, recovery passes, monitors, and continuous improvement cycles.
- From blind delegation to supervision: parent agents can list, steer, and stop running subagents, making delegation inspectable and governable.
- From one model to deliberation: /council can turn multi-model comparison into a repeatable pattern for architecture decisions and product trade-offs.
- From one machine to a mesh: Desktop multi-instance control suggests that Mercury, Hermes workstation, and remote nodes can become one agent roster.
- From browser clicking to API skill: HAR-derived API learning can convert repetitive website workflows into direct, faster, lower-friction calls.
- From context bloat to session retrieval: lean compaction reinforces the need for HVE to build durable memory outside a single active prompt.

3. Feature candidates for HVE Life OS

Hermes update	HVE opportunity	Luna architecture question	Priority
Bot Mode	Define named HVE agents: Time Steward, Physical Wealth Coach, Mental Wealth Guide, Social Wealth Connector, Financial Wealth Analyst, Auditor, and Release Manager.	How do we map bot identity, memory, tools, and permissions to HVE agent boundaries without creating overlap or unsafe autonomy?	P0
/loop	Run recurring checks: website health, dashboard refresh, CRM follow-up prompts, habit tracker audits, RAG ingestion quality, and weekly HVE briefings.	What is the smallest loop runner that can log state, stop cleanly, and expose evidence before taking action?	P0
Subagent control	Let Hermes-Coder delegate build/test/doc tasks while Luna and Hans can inspect or stop workstreams.	What supervision contract should every delegated task include: objective, files touched, evidence, stop condition, and risk level?	P0
/council	Use multi-model deliberation for architecture choices, sovereignty trade-offs, product positioning, and risk reviews.	Can we make Council outputs structured: agreement, disagreement, assumptions, risks, recommendation, and decision log?	P1
Multi-instance Desktop	Federate HVE agents across Hermes workstation, Mercury node, and future client/local nodes.	What should be local-only on Mercury versus routed to the workstation or cloud-hosted gateway?	P1

MCP overhaul	Package HVE tools as manageable integrations rather than hand-configured one-offs.	Can the HVE tool registry expose health, ownership, scopes, credentials, and install provenance?	P1
Plugins	Build HVE-specific capabilities as plugins: tracker views, agent roster, evidence ledger, data importers, approval workflows.	Which HVE behaviours belong in plugins versus scripts versus core application code?	P1
HAR-derived API client	Convert repetitive web workflows into direct API interactions after browser learning.	What guardrails are needed before Hermes can store and replay learned API requests?	P2
Lean compaction	Keep long-running agent work stable while preserving anchors and retrieving history from durable memory.	How should HVE split memory among session search, durable collector, Obsidian/markdown vault, and vector/RAG index?	P0

4. Recommended near-term experiments

Experiment A: HVE Agent Roster v0.1

Create a small roster of named HVE bots with explicit scopes, tools, memory boundaries, and "human approval required" rules. Start with Auditor, Builder, Researcher, and Steward before creating pillar-specific agents.

Experiment B: Sovereignty Loop Runner

Implement one safe recurring loop: check the HVE website build/deploy status, summarize the latest result, and produce an evidence-backed report. No automatic changes in v0.1.

Experiment C: Delegation Contract Template

Standardize how Hermes-Coder spawns subagents: task, files, constraints, stop condition, evidence required, and what to return to Luna/Hans.

Experiment D: Council Decision Record

Use /council for one architecture decision: "What is the best durable shared context layer for HVE Life OS on Mercury?" Capture outputs into a decision record.

Experiment E: Durable Memory Split

Test lean compaction plus external session search by storing anchors in markdown and retrieval artifacts in the shared context layer. Evaluate whether long sessions remain coherent.

5. Risks and guardrails

Risk	Why it matters	Guardrail
Agent sprawl	Bot Mode can make it easy to create too many agents with overlapping accountability.	Maintain a canonical HVE agent registry with owner, scope, tools, memory, and decommission criteria.
Runaway loops	Recurring tasks can become noisy, expensive, or unsafe if they act without bounds.	Every loop needs interval, max runs, stop condition, evidence log, and human approval for external changes.
Tool overreach	MCP and HAR-derived API use may grant more capability than intended.	Use least-privilege credentials, read-only mode first, explicit approval for writes, and auditable tool usage.
Cross-machine ambiguity	Multi-instance control can blur where data lives and where work executes.	Tag each agent with execution node, data boundary, and allowed network surfaces.

Memory confusion	Lean compaction and retrieval can hide what the agent actually knows versus what it recovered.	Separate active context, durable notes, retrieved evidence, and assumptions in every report.
-------------------------	--	--

6. Suggested ask for Luna

Architecture tasking prompt

Luna, please review the Hermes Aug 10 to 16 feature direction and produce an HVE Life OS technical adoption plan. Focus on Bot Mode, /loop, subagent supervision, Council Mode, multi-instance routing, MCP/plugin packaging, and lean compaction. For each, recommend: use now, prototype, monitor, or avoid. Include a minimal Mercury-compatible implementation path and identify the data, permission, logging, and rollback model required.

7. Proposed Luna decision record format

Decision area	Recommendation	Evidence required	Owner / next action
Bot roster	Prototype 4 agents maximum	Agent registry, permission matrix, sample conversations	Luna defines schema; Hermes-Coder implements
Recurring loops	Prototype read-only loop first	Run log, stop condition, false positive rate	Hermes-Coder builds one website/deploy monitor
Subagent control	Adopt contract before scaling delegation	Live transcript, task result, files changed, rollback note	Luna writes delegation spec
Council Mode	Use for architecture decisions only at first	Structured council output and decision record	Hans selects first decision question
MCP/plugins	Inventory current tools before adding more	Tool list, health, scopes, secret source, owner	Luna creates tool registry
Memory	Align with shared context layer v1	Anchor format, retrieval test, session export policy	Luna maps memory tiers

8. Sources and grounding notes

- Primary input: Hans Westphal supplied update text summarizing Hermes changes observed during Aug 10 to 16, 2026.
- External verification: GitHub release page for NousResearch/hermes-agent v2026.8.16 says the release was dated Aug 16, 2026 and rolled up fixes and improvements across Desktop, CLI, gateway, MCP health checks/deep links, /loop completion, cron, auth, and installer reliability.
- External context: Nous Research Hermes Agent product page describes Hermes as an open-source agent with persistent memory, scheduling, delegation, web/browser automation, tool use, multi-model reasoning, and isolated sandboxing.
- Internal HVE alignment: HVE Life OS Agent Organization Charter v0.3 defines the operating model "M365 Copilot thinks. Luna designs. Hermes builds. Mercury proves. Hans decides." and emphasizes local-first sovereignty across Time, Physical, Mental, Social, and Financial Wealth.
- Important caveat: Some items in Hans' supplied update appear to be pre-release or fast-moving features. Luna should verify exact CLI syntax, availability, and configuration behaviour against the installed Hermes build before implementation.

Appendix A: Fast feature map

Feature	Best HVE use	First proof point
Bot Mode	Agent roster tied to HVE roles	Four bot profiles with scoped skills
/loop	Recurring operational monitors	Read-only deploy/status monitor
Subagent control	Supervised build workers	List, steer, stop test task
/council	Architecture decision support	One decision record
Multi-instance Desktop	Mercury + workstation routing	Node-labelled roster
MCP overhaul	Tool registry and health checks	One MCP server with clear scope
Plugins	HVE-specific app layer	Evidence ledger plugin outline
HAR API skill	Workflow acceleration	One read-only website/API discovery test
Lean compaction	Long-running session stability	Session export plus retrieval test